

JAVA PRACTICAL RECORD

Experiment No.	04
Date	18/09/2026
Program	OrderDemo
Concept	Inheritance and Method Implementation

1. Objective / Aim

To design and implement a Java program demonstrating inheritance using Customer, Order, SpecialOrder, and NormalOrder classes, and to perform different order-related operations.

2. System Requirements

Hardware: Computer/Laptop with basic processing capability.

Software: Java Development Kit (JDK) 8 or above and any Java IDE or command-line terminal.

3. Algorithm / Logic Description

1. Start the program.
2. Create a **Customer** class containing name, location, sendOrder(), and receiveOrder().
3. Create an **Order** class containing date, number, confirm(), and close().
4. Create **SpecialOrder** by inheriting the Order class and define dispatch().
5. Create **NormalOrder** by inheriting the Order class and define dispatch() and receive().
6. Create a Customer object and assign its name and location.
7. Create SpecialOrder and NormalOrder objects and assign their date and order number.
8. Call the appropriate methods to confirm, dispatch, receive, and close the orders.
9. Display all customer and order details.
10. Stop the program.

4. UML Class Diagram

Customer	Order	SpecialOrder	NormalOrder
- name : String - location : String	- date : String - number : String	inherits Order	inherits Order
+ sendOrder() : void + receiveOrder() : void	+ confirm() : void + close() : void	+ dispatch() : void	+ dispatch() : void + receive() : void

Inheritance relationships: **SpecialOrder** → **Order** and **NormalOrder** → **Order**. Both child classes use the inherited date, number, confirm(), and close() members and add their own operations.

5. Source Code

```
// Customer class
class Customer {
    String name;
    String location;

    void sendOrder() {
        System.out.println(name + " sent the order.");
    }

    void receiveOrder() {
        System.out.println(name + " received the order.");
    }
}

// Order class
class Order {
    String date;
    String number;

    void confirm() {
        System.out.println("Order " + number + " is confirmed.");
    }

    void close() {
        System.out.println("Order " + number + " is closed.");
    }
}

// SpecialOrder inherits Order
class SpecialOrder extends Order {

    void dispatch() {
        System.out.println("Special order " + number + " is dispatched.");
    }
}

// NormalOrder inherits Order
class NormalOrder extends Order {

    void dispatch() {
        System.out.println("Normal order " + number + " is dispatched.");
    }

    void receive() {
        System.out.println("Normal order " + number + " is received.");
    }
}

// Main class
public class OrderDemo {
    public static void main(String[] args) {

        // Creating Customer object
        Customer customer = new Customer();
        customer.name = "Ravi";
        customer.location = "Vizianagaram";

        System.out.println("Customer Details");
        System.out.println("Name      : " + customer.name);
        System.out.println("Location : " + customer.location);
        customer.sendOrder();
        customer.receiveOrder();

        System.out.println();

        // Creating SpecialOrder object
        SpecialOrder special = new SpecialOrder();
        special.date = "18-09-2026";
        special.number = "S101";

        System.out.println("Special Order");
        System.out.println("Date   : " + special.date);
        System.out.println("Number : " + special.number);
        special.confirm();
        special.dispatch();
        special.close();

        System.out.println();

        // Creating NormalOrder object
```

```

        NormalOrder normal = new NormalOrder();
        normal.date = "18-09-2026";
        normal.number = "N101";

        System.out.println("Normal Order");
        System.out.println("Date   : " + normal.date);
        System.out.println("Number : " + normal.number);
        normal.confirm();
        normal.dispatch();
        normal.receive();
        normal.close();
    }
}

```

6. Compilation & Execution Commands

```
javac OrderDemo.java
```

```
java OrderDemo
```

7. Output Screen

```

Customer Details
Name       : Ravi
Location   : Vizianagaram
Ravi sent the order.
Ravi received the order.

Special Order
Date      : 18-09-2026
Number    : S101
Order S101 is confirmed.
Special order S101 is dispatched.
Order S101 is closed.

Normal Order
Date      : 18-09-2026
Number    : N101
Order N101 is confirmed.
Normal order N101 is dispatched.
Normal order N101 is received.
Order N101 is closed.

```

8. Result

The Java program was successfully implemented and executed. The program demonstrates inheritance through SpecialOrder and NormalOrder extending Order, while displaying the customer and order operations successfully.